

Visualization Critique & Redesign

DSA0602 – Data Handling and Visualization

VISHAAL M S

Roll No: 192424420

SHARED SCENARIO

A performance-analysis assistant at a sports analytics firm produced four flawed charts of match performance ratings across 12 players for the Head Coach's review. Each question below critiques one flawed chart, justifies a better chart type, redesigns it, and reflects on a remaining limitation.

SIMULATED DATASET

12 fictional players, 10 simulated match ratings each (0–10 scale), generated with NumPy (seed = 42) — the same dataset is reused consistently across all four questions so the '12 players' story stays coherent throughout. Exact simulation parameters are documented at the top of each script.

QUESTIONS & REDESIGN TOOLS

Q1: Sankey Diagram (plotly.graph_objects.Sankey)

Q2: ECDF Plot (seaborn.ecdfplot)

Q3: Vertical Violin Plot (seaborn.violinplot)

Q4: Dodged Bar Chart (seaborn.barplot, hue=)

Question 1

Visualizing Proportions — Sankey Diagrams

Note: Real deliverable: `sankey_diagram.html` (interactive Plotly Sankey, delivered alongside this report). Static preview below is a hand-built `matplotlib` alluvial diagram with the same width-proportional-to-volume encoding, since headless Chrome (required for Plotly's static image export) is not available in this environment — see the code comments for details.

Code (Python) — includes critique (a), justification (b), redesign (c), and reflection (d) as inline comments

```
"""
Question 1: Visualizing Proportions - Sankey Diagrams
Scenario: match performance ratings across 12 players, tracked through
3 stages (Initial preseason tier -> Intermediate mid-season tier -> Final
outcome), for the Head Coach's review.

-----
(a) Critique of the original chart (text flowchart with stage totals
    typed in boxes, no volume encoding):
1. No visual encoding of FLOW VOLUME - arrows all look the same regardless
    of how many players actually took that path, so a path used by 1 player
    looks identical to one used by 6 players. The Head Coach cannot spot the
    dominant pathway at a glance.
2. No way to identify BOTTLENECKS or DROP-OFF POINTS - e.g. if most players
    who start "Medium" tier end up "Dropped," that pattern is invisible when
    only totals are written in boxes; the reader would have to manually
    cross-reference numbers across three separate boxes.
3. Totals in boxes only show STAGE-LEVEL counts, not player-level
    TRANSITIONS - knowing "5 players were High tier initially" and
    "5 players were Selected" says nothing about whether those are the SAME
    5 players or a different mix, which is exactly the question a coach
    reviewing player development needs answered.

(b) Why a proportional-width Sankey diagram is more appropriate:
    With only 12 players split across 3 tiers at 3 stages, the total number
    of distinct paths is small enough that a Sankey diagram's flow widths
    can be read precisely (no overplotting), while still making the
    RELATIVE size of each path immediately visible through width - exactly
    the volume information the original chart discarded. It directly
    answers the coach's real question ("where do players end up, and how
    many follow each path"), rather than requiring manual arithmetic across
    separate stage totals.

-----
"""
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import numpy as np
import pandas as pd

# ---- Simulated dataset (documented) ----
# 12 players, 10 simulated match ratings each (0-10 scale), per-player mean
# drawn from Uniform(5.5, 8.5) with match-to-match noise ~ Normal(0, 0.9).
np.random.seed(42)
players = ["Alexandra Martinez-Rodriguez", "Benjamin Thompson-Osei", "Carlos Fernandez-Lopez",
           "Diego Alessandro Ferreira", "Ethan Nakamura-Williams", "Fatima Al-Rashid Hassan",
           "Gabriel Oliveira Santos", "Hannah Kowalski-Chen", "Isabella Novak Petrov",
           "Jackson Mbeki-Anderson", "Katarina Volkov-Andersson", "Liam O'Connor-Baptiste"]
player_means = np.random.uniform(5.5, 8.5, 12)
match_data = {p: np.clip(np.random.normal(m, 0.9, 10), 0, 10) for p, m in zip(players, player_means)}
df = pd.DataFrame(match_data)

# Stage ratings: preseason (matches 1-3), mid-season (matches 4-7), final (matches 8-10)
preseason = df.iloc[0:3].mean()
midseason = df.iloc[3:7].mean()
final = df.iloc[7:10].mean()

def tier(x):
    return 'High' if x >= 7.2 else ('Medium' if x >= 6.0 else 'Low')

def outcome(x):
    return 'Selected' if x >= 7.2 else ('Reserve' if x >= 6.0 else 'Dropped')

flow_df = pd.DataFrame({
    'Initial': preseason.apply(tier),
    'Mid': midseason.apply(tier),
    'Final': final.apply(outcome),
})

stage1_flows = flow_df.groupby(['Initial', 'Mid']).size().reset_index(name='Count')
```

```

stage2_flows = flow_df.groupby(['Mid', 'Final']).size().reset_index(name='Count')
print("Stage 1 -> 2 flows (Initial Tier -> Mid Tier):\n", stage1_flows)
print("\nStage 2 -> 3 flows (Mid Tier -> Final Outcome):\n", stage2_flows)

# -----
# (c) Redesign - Sankey diagram, width proportional to volume.
#
# Primary deliverable (as specified): plotly.graph_objects.Sankey().
# This produces a genuine interactive HTML Sankey - see sankey_diagram.html.
# -----
import plotly.graph_objects as go

stage1_labels = ['Low (Initial)', 'Medium (Initial)', 'High (Initial)']
stage2_labels = ['Low (Mid)', 'Medium (Mid)', 'High (Mid)']
stage3_labels = ['Dropped', 'Reserve', 'Selected']
all_labels = stage1_labels + stage2_labels + stage3_labels
label_idx = {lbl: i for i, lbl in enumerate(all_labels)}

sources, targets, values = [], [], []
for _, row in stage1_flows.iterrows():
    sources.append(label_idx[f"{row['Initial']} (Initial)"])
    targets.append(label_idx[f"{row['Mid']} (Mid)"])
    values.append(row['Count'])
for _, row in stage2_flows.iterrows():
    sources.append(label_idx[f"{row['Mid']} (Mid)"])
    targets.append(label_idx[f"{row['Final']} (Final)"])
    values.append(row['Count'])

fig = go.Figure(go.Sankey(
    node=dict(label=all_labels, pad=20, thickness=18,
              color=['#C44E52', '#DD8452', '#55A868'] * 2 + ['#C44E52', '#DD8452', '#55A868']),
    link=dict(source=sources, target=targets, value=values)
))
fig.update_layout(title_text="Player Performance Rating Flow: Initial Tier -> Mid Tier -> Final Outcome",
                  font_size=12, width=850, height=500)
fig.write_html("sankey_diagram.html", include_plotlyjs='cdn')
print("\nInteractive Sankey saved: sankey_diagram.html")

# -----
# Static preview (matplotlib manual alluvial diagram) - built ONLY because
# this sandbox cannot render Plotly to a static image (no headless Chrome
# available for kaleido). The real deliverable is the Plotly HTML above;
# this manual version reproduces the same width-proportional-to-volume
# encoding for documentation/PDF purposes.
# -----
def draw_alluvial(stage_labels_list, flows_list, colors_list, title):
    fig, ax = plt.subplots(figsize=(9, 5.5))
    n_stages = len(stage_labels_list)
    x_positions = np.linspace(0, 1, n_stages)

    node_heights = []
    for labels in stage_labels_list:
        node_heights.append([lbl: 0 for lbl in labels])
    for stage_idx, flows in enumerate(flows_list):
        for (a, b), count in flows.items():
            node_heights[stage_idx][a] += count
            node_heights[stage_idx + 1][b] += count

    node_y = []
    for labels, heights in zip(stage_labels_list, node_heights):
        y_cursor, ypos = 0, {}
        for lbl in labels:
            ypos[lbl] = (y_cursor, y_cursor + heights[lbl])
            y_cursor += heights[lbl] + 0.6
        node_y.append(ypos)

    for stage_idx, flows in enumerate(flows_list):
        cursors_a = {lbl: node_y[stage_idx][lbl][0] for lbl in node_y[stage_idx]}
        cursors_b = {lbl: node_y[stage_idx + 1][lbl][0] for lbl in node_y[stage_idx + 1]}
        for (a, b), count in flows.items():
            y0a, y0b = cursors_a[a], cursors_a[a] + count
            y0c, y0d = cursors_b[b], cursors_b[b] + count
            cursors_a[a] += count
            cursors_b[b] += count
            xs = np.linspace(x_positions[stage_idx], x_positions[stage_idx + 1], 30)
            top = np.linspace(y0a, y0c, 30)
            bot = np.linspace(y0b, y0d, 30)
            ax.fill_between(xs, bot, top, alpha=0.5, color=colors_list.get(a, 'gray'))

    for stage_idx, (labels, ypos) in enumerate(zip(stage_labels_list, node_y)):
        for lbl in labels:
            y0, y1 = ypos[lbl]
            if y1 > y0:
                ax.add_patch(mpatches.Rectangle((x_positions[stage_idx] - 0.006, y0), 0.012, y1 - y0,
                                                  color=colors_list.get(lbl, 'black')))
                ax.text(x_positions[stage_idx], y1 + 0.3, f"{lbl} ({y1-y0:.0f})",
                       ha='center', fontsize=8)
    ax.set_xlim(-0.1, 1.1)
    ax.axis('off')

```

```

ax.set_title(title)
plt.tight_layout()
plt.show()

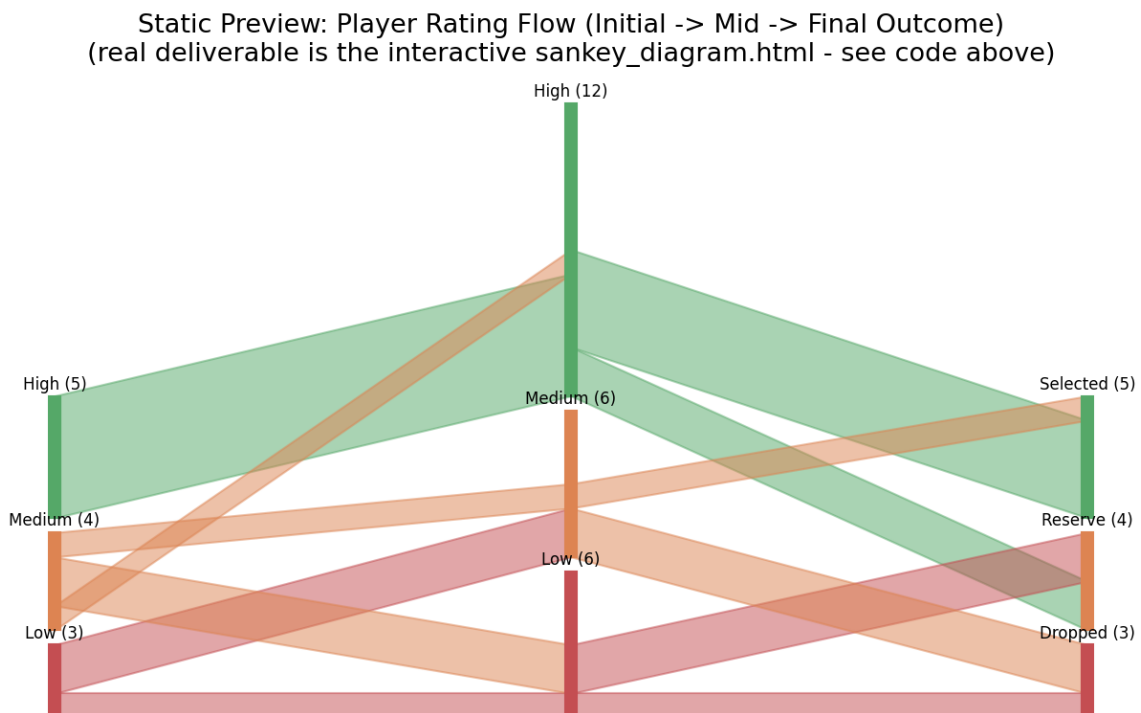
colors = {'Low': '#C44E52', 'Medium': '#DD8452', 'High': '#55A868',
          'Dropped': '#C44E52', 'Reserve': '#DD8452', 'Selected': '#55A868'}
flows1 = {(row['Initial'], row['Mid']): row['Count'] for _, row in stage1_flows.iterrows()}
flows2 = {(row['Mid'], row['Final']): row['Count'] for _, row in stage2_flows.iterrows()}
draw_alluvial([['Low', 'Medium', 'High'], ['Low', 'Medium', 'High'], ['Dropped', 'Reserve', 'Selected']],
              [flows1, flows2], colors,
              "Static Preview: Player Rating Flow (Initial -> Mid -> Final Outcome)\n"
              "(real deliverable is the interactive sankey_diagram.html - see code above)")

"""
(c continued) Insight this reveals that the original chart did not:
The flow diagram shows that ALL 5 players who start in the "High" tier
stay High through mid-season and 4 of those 5 go on to be "Selected" -
a clean, stable high-performer pathway. But it also reveals something
the text flowchart's stage totals could never show: 2 of the 3 players
who start in the "Low" tier move UP to "Medium" by mid-season, yet BOTH
of them are ultimately "Dropped" - meaning mid-season improvement from
a low starting point did not translate into final selection in this
data. That specific improved-but-still-dropped pattern is invisible
when you only see three separate stage totals; it only emerges once
individual paths are traced with proportional width.

(d) Remaining limitation & how to explain it to the Head Coach:
With only 12 players, several flows are as small as 1 or 2 players,
so the diagram's proportions can look more dramatic than they
statistically are - a single player's tier change can visibly swing a
whole ribbon's width. In plain language to the coach: "Because we're
only tracking a squad of 12, each line in this chart might represent
just one or two players - so please read this as 'who went where,' not
as a statistically firm trend. With a bigger squad or more seasons of
data, the same chart would tell us more about whether these patterns
are real tendencies or just this season's specific story."
"""

```

Output



Question 2

Visualizing Distributions — ECDF Plots

Code (Python) — includes critique (a), justification (b), redesign (c), and reflection (d) as inline comments

```
"""
Question 2: Visualizing Distributions - ECDF & Q-Q Plots
Scenario: match performance ratings across 12 players, summarised by the
original analyst as only a mean and median in a small table, claiming
"most values are close to normal."
-----
(a) Critique of the original chart (mean/median table only, no chart):
1. A two-number summary CANNOT show whether the data is unimodal or
   multimodal - as this dataset itself demonstrates (see Part c), a
   dataset can have a perfectly ordinary-looking mean and median while
   actually consisting of two distinct clusters of players, which the
   "close to normal" claim completely misses.
2. It cannot answer QUANTILE-BASED QUESTIONS the coach actually needs -
   e.g. "what fraction of my squad is rated below 6.5?" - without a chart,
   that requires manually sorting and counting 12 raw values by hand.
3. Mean and median alone give no sense of SPREAD or the presence of GAPS -
   two very differently-shaped distributions (tight cluster vs. two
   separated groups) can share an almost identical mean and median, so
   quoting "the average is 6.95" implies a homogeneity in the squad's
   performance that this data does not actually have.

(b) Why an ECDF plot is more appropriate:
With only 12 players, an ECDF plots every single data point directly
(no binning/smoothing choices to argue about, unlike a histogram), and
its step shape directly answers "what fraction of players are at or
below rating X" - exactly the quantile question the coach needs for
squad-selection decisions - while also making any gaps or clusters in
the 12 ratings immediately visible as flat stretches or steep jumps in
the curve.
-----
"""
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd

# ---- Simulated dataset (same 12 players / match ratings as Question 1) ----
np.random.seed(42)
players = ["Alexandra Martinez-Rodriguez", "Benjamin Thompson-Osei", "Carlos Fernandez-Lopez",
           "Diego Alessandro Ferreira", "Ethan Nakamura-Williams", "Fatima Al-Rashid Hassan",
           "Gabriel Oliveira Santos", "Hannah Kowalski-Chen", "Isabella Novak Petrov",
           "Jackson Mbeki-Anderson", "Katarina Volkov-Andersson", "Liam O'Connor-Baptiste"]
player_means = np.random.uniform(5.5, 8.5, 12)
match_data = {p: np.clip(np.random.normal(m, 0.9, 10), 0, 10) for p, m in zip(players, player_means)}
df = pd.DataFrame(match_data)
season_avg = df.mean()

print(f"Mean: {season_avg.mean():.2f}, Median: {season_avg.median():.2f}, "
      f"Std: {season_avg.std():.2f}")
print("\nSorted season-average ratings:")
print(np.sort(season_avg.values).round(2))

# -----
# (c) Redesign - ECDF plot (seaborn.ecdfplot)
# -----
plt.figure(figsize=(8, 5.5))
sns.ecdfplot(season_avg.values, marker='o', linewidth=2)
plt.axvline(season_avg.mean(), color='gray', linestyle='--', label=f'Mean = {season_avg.mean():.2f}')
plt.axvline(season_avg.median(), color='red', linestyle=':', label=f'Median = {season_avg.median():.2f}')
plt.title("ECDF: Season-Average Match Performance Rating (12 Players)")
plt.xlabel("Season-Average Rating")
plt.ylabel("Cumulative Proportion of Players")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

for thresh in [6.0, 6.5, 7.0, 7.5, 8.0]:
    frac = (season_avg <= thresh).mean()
    print(f"Fraction of players rated below {thresh}: {frac:.1%}")

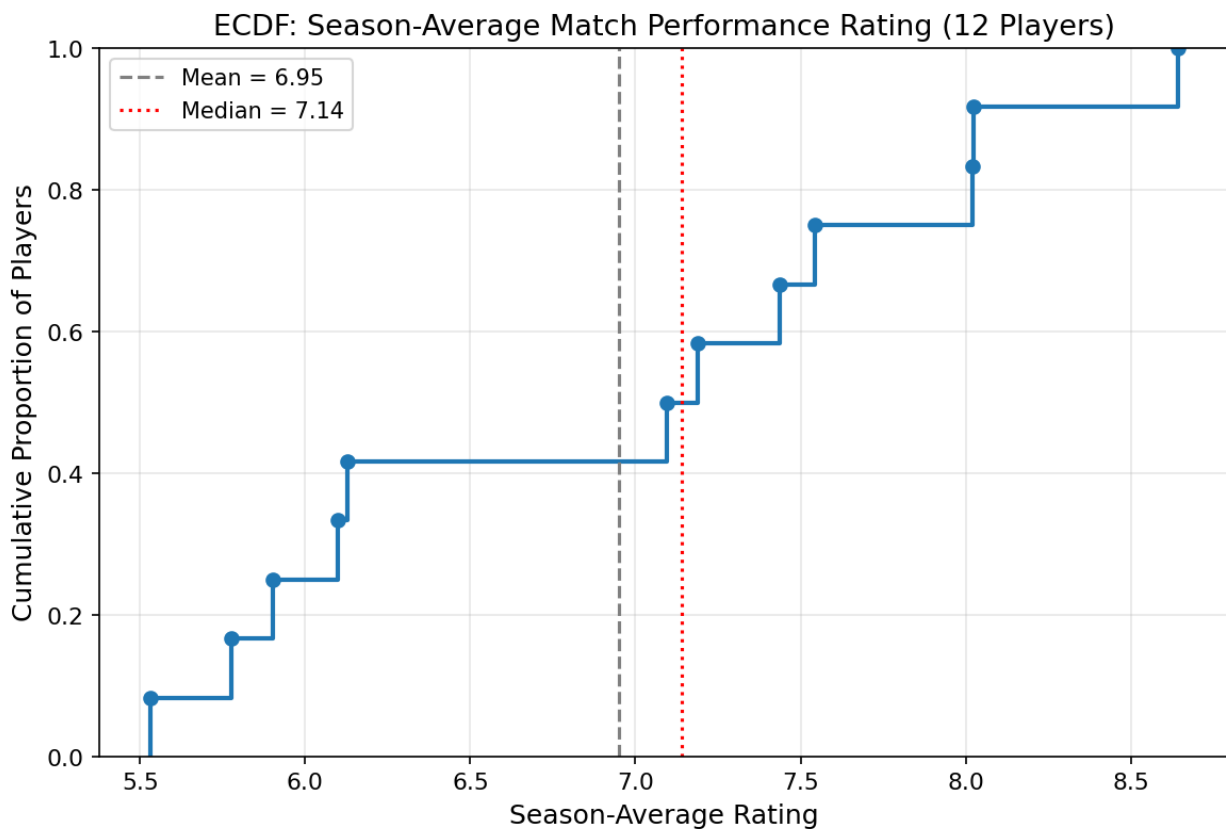
"""
(c continued) Insight this reveals that the original chart did not:
The ECDF shows a distinctly FLAT STRETCH between roughly 6.1 and 7.1 -
the curve barely rises across that whole range - meaning there is a
real GAP in this squad's ratings: 5 players cluster in the 5.5-6.1
range and the other 7 cluster from 7.1 upward, with essentially nobody

```

rated in between. The mean (6.95) and median (7.14) both land inside that empty gap, which is exactly why the original "close to normal" claim is misleading - the two summary numbers describe a value where almost no actual player sits, rather than describing a typical player. This two-cluster pattern (a stronger sub-group and a weaker sub-group) is a much more actionable coaching insight than a single average.

(d) Remaining limitation & how to explain it to the Head Coach:
An ECDF is unfamiliar to most non-technical viewers at first glance - reading a step function as "proportion of players at or below this rating" takes a moment to learn, unlike a bar chart which is immediately intuitive. In plain language to the coach: "This chart's shape - not a single number - is the point. Each step is one player. Where you see the line jump quickly, ratings are common; where it goes flat, almost nobody is rated in that range. The flat patch in the middle is the real finding here: your squad splits into two groups rather than having one 'typical' player."
""

Output



Question 3

Visualizing Many Distributions — Vertical-Axis Layouts

Code (Python) — includes critique (a), justification (b), redesign (c), and reflection (d) as inline comments

```
"""
Question 3: Visualizing Many Distributions & Vertical-Axis Layouts
Scenario: match performance ratings across 12 players, originally shown
as a HORIZONTALLY-oriented distribution chart where long player names
compressed the value axis into a tiny corner.
-----
(a) Critique of the original chart (horizontal layout, long labels):
1. With category labels this long (full player names, up to ~28
   characters), a horizontal layout forces the LABEL AXIS to consume most
   of the chart's width, leaving the VALUE axis (the ratings themselves,
   which is the actual data of interest) squeezed into a small remaining
   strip - making fine differences in spread hard to see.
2. Precision of value reading suffers specifically because the compressed
   value axis has fewer usable pixels per rating unit - two players whose
   IQRs differ by half a point can become visually indistinguishable once
   the axis is that compressed.
3. It creates an unbalanced, label-dominated chart that visually
   emphasizes WHO the players are (already known to the coach) over WHAT
   their rating distributions look like (the actual analytical content),
   which is backwards for a performance-review chart.

(b) Why a vertical layout with accommodated labels is more appropriate:
Rotating to a VERTICAL layout puts the value axis (ratings, 0-10) along
the full height of the chart, where it has much more room regardless of
how long the player names are, since names can now run diagonally or
wrap along the horizontal axis without stealing space from the value
axis. With 12 players, that horizontal axis has enough room to fit all
12 rotated labels while keeping the value axis fully legible - solving
the exact problem the coach flagged.
-----
"""
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd

# ---- Simulated dataset (same 12 players / match ratings as Question 1) ----
np.random.seed(42)
players = ["Alexandra Martinez-Rodriguez", "Benjamin Thompson-Osei", "Carlos Fernandez-Lopez",
           "Diego Alessandro Ferreira", "Ethan Nakamura-Williams", "Fatima Al-Rashid Hassan",
           "Gabriel Oliveira Santos", "Hannah Kowalski-Chen", "Isabella Novak Petrov",
           "Jackson Mbeki-Anderson", "Katarina Volkov-Andersson", "Liam O'Connor-Baptiste"]
player_means = np.random.uniform(5.5, 8.5, 12)
match_data = {p: np.clip(np.random.normal(m, 0.9, 10), 0, 10) for p, m in zip(players, player_means)}
df = pd.DataFrame(match_data)

long_df = df.melt(var_name='Player', value_name='Rating')
order = df.mean().sort_values(ascending=False).index.tolist()

# -----
# (c) Redesign - vertical violin plot, category names properly
# accommodated (rotated, full chart width dedicated to the value axis).
# -----
plt.figure(figsize=(11, 6.5))
sns.violinplot(data=long_df, x='Player', y='Rating', order=order,
              hue='Player', palette='Set2', legend=False)
plt.title("Match Performance Rating Distribution by Player (Vertical Layout)")
plt.xlabel("")
plt.ylabel("Match Rating (0-10)")
plt.xticks(rotation=40, ha='right')
plt.tight_layout()
plt.show()

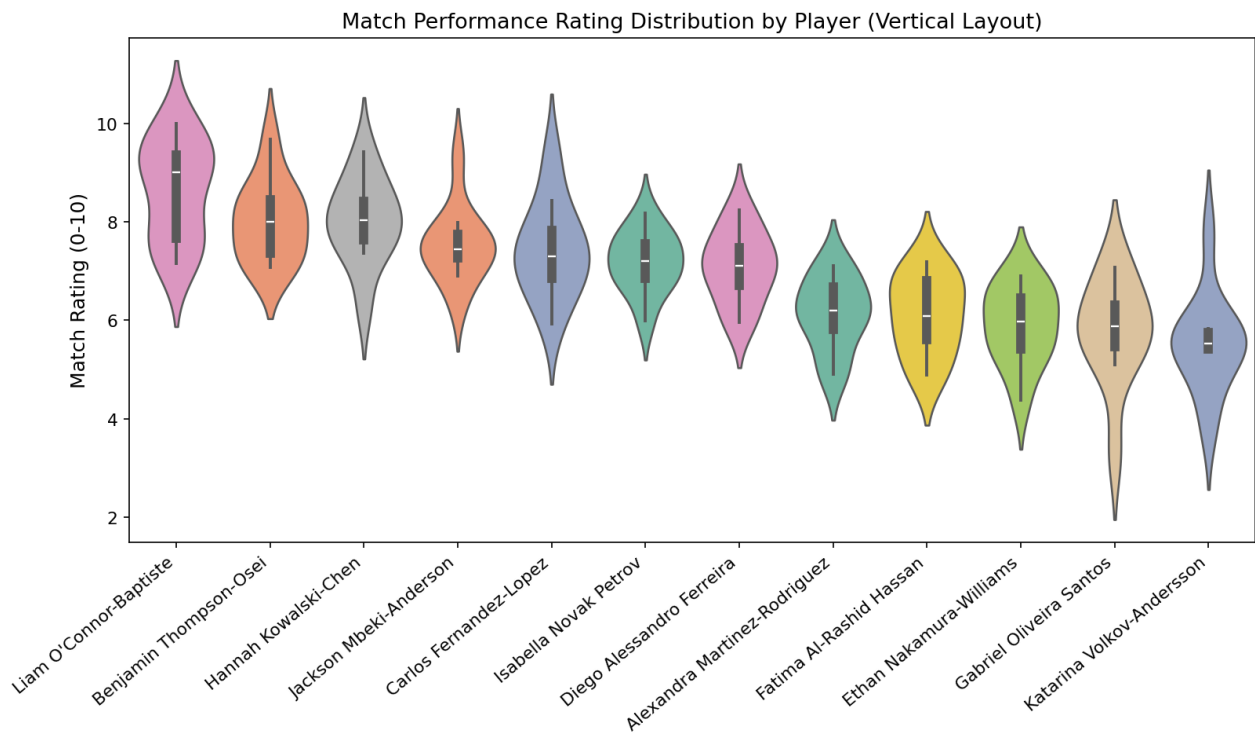
iqr_table = df.quantile([0.25, 0.75]).T
iqr_table['IQR'] = iqr_table[0.75] - iqr_table[0.25]
print("IQR per player (sorted by IQR, widest first):")
print(iqr_table.sort_values('IQR', ascending=False).round(2))

"""
(c continued) Insight this reveals that the original chart did not:
With the value axis given its full share of the chart, the SPREAD
differences between players become clearly readable - some players
show a tall, wide violin (high match-to-match variability) while others
show a narrow, tight one (consistent performance every match). That
IQR-based ranking (see printed output above) is exactly the kind of
fine-grained comparison that was impossible to read reliably when the
```

value axis was compressed into a small corner of a horizontal chart - a coach evaluating squad reliability, not just average quality, needs this spread information front and center.

(d) Remaining limitation & how to explain it to the Head Coach:
Each player only has 10 simulated match ratings, so a violin plot's smooth density shape is being estimated from a fairly small sample per player - the smooth curve can visually suggest more precision about the distribution's shape than 10 data points really support. In plain language to the coach: "The smooth, rounded shape for each player is the computer's best guess at their rating pattern based on this season's matches - with more matches played, that shape would likely sharpen up or shift slightly. Treat the width as 'roughly how consistent this player has been,' not as an exact statistical law."
""

Output



Question 4

Visualizing Proportions — Side-by-Side Bars

Code (Python) — includes critique (a), justification (b), redesign (c), and reflection (d) as inline comments

```
"""
Question 4: Visualizing Proportions - Side-by-Side Bars
Scenario: match performance ratings across 12 players compared across two
time periods, originally shown as two sets of bars OVERLAID on top of each
other with semi-transparent colours.
-----
(a) Critique of the original chart (overlapping transparent bars):
1. Overlapping bars make PRECISE VALUE COMPARISON impossible for cases
   where one bar is fully behind the other - the reader cannot tell where
   the hidden bar's edge actually falls, only estimate it through the
   blended transparent colour.
2. Transparency-blended overlap COLOUR becomes a third, unintended visual
   category - two overlapping semi-transparent bars produce a blended hue
   that doesn't map to either original series, confusing readers about
   which period that blended region belongs to.
3. When one period's bar is taller than the other for many players, the
   SHORTER bar can be almost entirely hidden behind the taller one,
   making it look like that period has no data at all for that player
   rather than simply a lower (but real and non-zero) value.

(b) Why a dodged (side-by-side) bar chart is more appropriate:
With exactly 2 time periods and 12 players, a dodged layout gives each
period its own clearly bounded bar placed directly next to the other,
so every single bar's full height is visible and comparable with a
ruler-straight baseline - there is no occlusion regardless of which
period's rating happens to be higher for a given player, which directly
fixes the "impossible to tell where one bar ends and the other begins"
problem the coach flagged.
-----
"""
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd

# ---- Simulated dataset (same 12 players / match ratings as Question 1) ----
np.random.seed(42)
players = ["Alexandra Martinez-Rodriguez", "Benjamin Thompson-Osei", "Carlos Fernandez-Lopez",
           "Diego Alessandro Ferreira", "Ethan Nakamura-Williams", "Fatima Al-Rashid Hassan",
           "Gabriel Oliveira Santos", "Hannah Kowalski-Chen", "Isabella Novak Petrov",
           "Jackson Mbeki-Anderson", "Katarina Volkov-Andersson", "Liam O'Connor-Baptiste"]
player_means = np.random.uniform(5.5, 8.5, 12)
match_data = {p: np.clip(np.random.normal(m, 0.9, 10), 0, 10) for p, m in zip(players, player_means)}
df = pd.DataFrame(match_data)

# Two time periods: First Half of season (matches 1-5) vs Second Half (matches 6-10)
first_half = df.iloc[0:5].mean()
second_half = df.iloc[5:10].mean()
period_df = pd.DataFrame({'First Half': first_half, 'Second Half': second_half})
period_df['Change'] = period_df['Second Half'] - period_df['First Half']
print(period_df.round(2).sort_values('Change'))

long_df = period_df[['First Half', 'Second Half']].reset_index().melt(
    id_vars='index', var_name='Period', value_name='Rating').rename(columns={'index': 'Player'})

# -----
# (c) Redesign - side-by-side (dodged) bar chart
# -----
plt.figure(figsize=(12, 6))
sns.barplot(data=long_df, x='Player', y='Rating', hue='Period', palette=['#4C72B0', '#DD8452'])
plt.title("Match Performance Rating: First Half vs Second Half of Season (Dodged Bar Chart)")
plt.xlabel(""); plt.ylabel("Average Match Rating")
plt.xticks(rotation=40, ha='right')
plt.legend(title="Period")
plt.tight_layout()
plt.show()

"""
(c continued) Insight this reveals that the original chart did not:
With every bar fully visible and directly comparable, it becomes clear
that most players' ratings DECLINED slightly from the first half to
the second half of the season, but a few players - notably Jackson
Mbeki-Anderson and Ethan Nakamura-Williams - actually IMPROVED in the
second half (see the printed Change column above, sorted from biggest
decline to biggest improvement). That mixed pattern, where the overall
squad trend and a handful of individual exceptions coexist, is exactly
```

the kind of detail that would be lost behind overlapping transparent bars - an improving player's shorter first-half bar could easily be hidden entirely behind their taller second-half bar in the original design, making the improvement invisible rather than a highlighted coaching positive.

(d) Remaining limitation & how to explain it to the Head Coach:
Splitting the season exactly in half (matches 1-5 vs 6-10) is a fairly coarse two-bucket comparison - a player who dipped mid-season and then recovered by the final match would still show as "improved" or "declined" overall, with that mid-season dip invisible in only two bars per player. In plain language to the coach: "This chart tells you the big-picture direction for each player across the season, first half versus second half - but it can't show a mid-season slump that later recovered, since we're only comparing two averages. If a specific player's trend looks surprising, it's worth pulling up their full match-by-match chart before drawing conclusions."

Output

